The Open Source Audit

A Capstone Project for OSS NGMC Course

Student Name	Registration Number
ASHISH SHARMA	24MIM10040

The Open-Source Audit: A Deep Dive into Git

1. Executive Context and Project Scope

Open Source Software isn't just a pile of random freebies anymore. It's at the heart of everything, from Kubernetes to the Linux kernel running supercomputers and powering most of what we do online. OSS flipped the script on how people share their ideas and work together.

This audit is all about Git. It's the distributed version control system that absolutely changed the way developers do their jobs. If you're a systems architect, you don't just use Git blindly you dig into how it works, look at its licensing, analyze its technical footprint, and see how the community rules keep things moving forward. If you want to really get why Git dominates, you need to look at what failed before, when companies controlled the tools.

2. Part A: Origin and Philosophy - The Genesis of Freedom

Where a tool came from says everything about its stability and the ethics of its community. Exploring Git's history isn't just a trip down memory lane, it helps you see if the project manages its technical debt and sets up for the long haul.

A1. The Origin Story: The BitKeeper Crisis

Before Git, Linux kernel devs shared patches by hand. In 2002, they got tired of that and switched to BitKeeper - a closed, proprietary system. That blew up in 2005. BitKeeper's company pulled the plug when they got spooked by reverse engineering. Linus Torvalds had a choice: go backwards or invent something totally new. He went all-in, built Git in weeks, made sure it was distributed, and slapped an open license on it. He didn't just fix version control he made sure nobody would get locked out ever again. Git exists because the old way was "broken, inaccessible, and restrictive."

A2. License Anatomy: Copyleft and the Four Freedoms

Git runs under the GNU General Public License (GPL) version 2. The Four Freedoms from the Free Software Foundation break it down:

1. You can run the program for any reason.

2. You can study and improve the code.

3. You can distribute copies.

4. You can share your changes.

Git gives you all those rights.

When you compare licenses:

Feature	GPL v2	MIT / Apache 2.0
Modification	Guaranteed	Guaranteed
Reciprocity	Mandatory (must share changes)	Optional (can keep changes private)
Corporate Silos	Not allowed, remains open	Allowed forks can become closed
So What?	Prevents code capture	Enables wider adoption, even proprietary

A3. The Ethical Landscape: Standing on Shoulders

Open source is all about building on what others have done.That lets innovation really take off.

Some people say software is pure knowledge like math formula and shouldn't belong to just one person or company. Giving folks transparency and letting others review and audit the code is the right call. On the other hand, proprietary models are useful for small markets, where big R&D dollars need to be recouped and open community projects might not be enough.

Big companies like Red Hat, Google, and Meta rely on FOSS and give back upstream. It's not about smothering innovation it's about getting the basics (OS, version control, databases) solid and letting everyone focus on building cool stuff at the application layer. Git's deep ties to the Linux kernel show off that technical transparency.

3. <u>Part B: Linux Footprint-Technical Implementation</u>

If you're a Senior Architect, you better know where Git gets installed security starts with knowing exactly what's running on your systems.

- **Installing Git**

On Ubuntu or RHEL, just use the package manager. Run *apt install git* or *dnf install git*. Git source folder:

- ***/usr/bin/git*** -- the actual binary
- ***/etc/gitconfig*** -- system-wide config

- ***~/.gitconfig*** -- user config

- ***/usr/share/doc/git*** -- docs and licenses

To check if it's there, use *dpkg -l git* (on Debian/Ubuntu) or *rpm -qi git* (on RHEL/CentOS).

- **User Context and Service Management**

Git's mostly a tool you run as yourself. On servers (with git-daemon), you check who's running it by using ps aux grep git or checking the UID in /etc/passwd.

Locally, Git doesn't sit around as a daemon. On servers, system manages git-daemon. You run:

- ***sudo systemctl*** start git-daemon

- ***sudo systemctl*** stop git-daemon

- ***sudo systemctl*** status git-daemon

- **Update Model**

Git stays safe because anyone can submit patches. When something goes wrong, the community jumps in to fix it, maintainers make sure those fixes reach older versions, and your package manager brings those updates straight to your system with a simple sudo apt upgrade. You don't have to sit around hoping a company will bail you out. It's all part of being connected to the bigger free and open-source software world.

4. Part C: The FOSS Ecosystem -- Dependencies and Governance

Git is right in the middle of a web of tech.

Dependencies

- Git needs:

  ○ zlib for compression ○ libcurl for network stuff (HTTP/HTTPS) ○ OpenSSL/LibreSSL for secure connections ○ Expat for XML parsing

- Modern Web: LAMP Stack and Beyond

Git doesn't just manage code it's behind the scenes in the DevOps revolution. LAMP runs your apps; Git manages change. Big projects like PyTorch, TensorFlow, and anything cloud-native all use Git, usually via GitHub or GitLab.

- Governance and Ownership

Git is governed by meritocracy. Stuff gets hashed out on the Git Mailing List. There's a Maintainer (right now, it's Junio C Hamano), but ultimately, consensus and technical merit win. Nobody owns Git, the community owned it.

5. Part D: Open Source vs. Proprietary - Critical Comparison

For a proper audit, you must compare. Here's how Git stacks up against Perforce (Helix Core) or Azure DevOps Server:

Verdict: Enterprise Suitability

Dimension	Git (Open Source)	Perforce (Proprietary)
Cost	$0 license; pay for hosting/admin	Per-user licensing; higher total cost
Security	Fully auditable; transparent code	Closed source; trust the vendor
Reliability	Distributed; each clone acts as backup	Centralized; server is single point
Freedom	Full control; modify, share, extend	Restricted by EULA

Control	Community-driven development	Corporate-controlled decisions

Honestly, Git is the smart move for enterprise. Low total cost, no vendor lock-in, tons of available expertise, and you can audit the system. Security checks -- You do them yourself.

I don't just recommend Git, i push people to make it better. Fixing your own tools is pure freedom and real professional power. Shell scripting takes that one step further with automation.

6. Shell Scripting Portfolio - The Automation Layer

Shell scripts turn Linux and FOSS tools into actual workflows that you can rely on.

- **Script 1: System Identity Report**

This script grabs your platform's details and links system health to the licensing that keeps it open.

```
#!/bin/bash
# Script 1: System Identity Report


# -------- Variables --------
```

```
STUDENT_NAME="ASHISH SHARMA"
SOFTWARE_CHOICE="Git"
```

```
# -------- System Information --------
DISTRO=$(hostnamectl | awk -F: '/Operating System/ {print $2}' | xargs)
KERNEL=$(uname -r)
USER_NAME=$(whoami) HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE=$(date)

# -------- Output -------- echo "   Open
Source Audit — System Report" echo " " echo
"Student Name : $STUDENT_NAME" echo
"Software     : $SOFTWARE_CHOICE" echo "----
-----" echo "Distribution : $DISTRO" echo
"Kernel       : $KERNEL" echo "User
: $USER_NAME" echo "Home Dir     :
$HOME_DIR" echo "Uptime       : $UPTIME"
echo "Date & Time  : $DATE" echo "----------
"
```

Script uses command substitution, grabs environment data, and spells out your system's roots.

- **Script 2: FOSS Package Inspector**

Checks if Git's installed, yanks its metadata, and tacks on a bit of philosophy.

```
#!/bin/bash
# Script 2: FOSS Package Inspector

# -------- Variable --------
```

```
PACKAGE="git"


# -------- Check Installation -------- if dpkg -s
"$PACKAGE" &>/dev/null; then     echo "$PACKAGE
is installed on this system."
```

```
    # Display key package details
    dpkg -s "$PACKAGE" | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed."
fi
# -------- Philosophy Note -------- case $PACKAGE in     git)
echo "Git: A decentralized version control system built for speed and
collaboration."
        ;;     python)          echo "Python: A community-driven
language focused on simplicity and readability."         ;;
firefox)          echo "Firefox: An open-source browser promoting a
free and open web."         ;;     vlc)          echo "VLC: A media
player that supports all formats without proprietary restrictions."
        ;;     apache2)          echo "Apache: A powerful open-source
web server that powers the internet."         ;;     *)          echo
"Unknown package: No philosophy note available."
        ;; esac
```

You can always check the provenance and story behind any tool you're running.

- **Script 3: Disk & Permission Auditor**

Audits key directories and checks ownership of Git's config file.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor


# -------- Directories to Audit --------
```

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin")


# -------- Software-Specific Path --------
GIT_CONFIG="$HOME/.gitconfig"
 echo "      Directory Audit Report"


# -------- Loop through directories --------
for DIR in "${DIRS[@]}"; do     if [ -d
"$DIR" ]; then
        # Extract permissions, owner, group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')


        # Get directory size (suppress errors)
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
                 echo "PATH       : $DIR"
echo "PERMISSIONS: $PERMS"         echo
"SIZE       : $SIZE"     else         echo
"Directory $DIR does not exist"     fi done


# -------- Software Specific Audit --------
echo "     Git Configuration Check"
 if [ -f "$GIT_CONFIG" ]; then     echo
"Git config found at: $GIT_CONFIG"


    # Show permissions and owner
    ls -l "$GIT_CONFIG" | awk '{print "Permissions: " $1 " | Owner: " $3}'
else
    echo "Git configuration file not found in user home directory." fi
```

Knowing who owns your config files is step one to real control.

- **Script 4: Log File Analyzer**

Searches logs for errors, retries if empty, and summarizes findings.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: ASHISH SHARMA
# Course: Open Source Software


# -------- Input Arguments --------
LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword = error
COUNT=0


# -------- Check if file argument is provided --------
if [ -z "$LOGFILE" ]; then     echo "Usage: $0
<logfile> [keyword]"     exit 1 fi
# -------- Check if file exists -------- if [ ! -f
"$LOGFILE" ]; then     echo "Error: File
'$LOGFILE' does not exist."     exit 1 fi
# -------- Wait until file has content --------
until [ -s "$LOGFILE" ]; do     echo "Waiting for
log file to contain data..."     sleep 2 done


# -------- Read file line by line -------- while
IFS= read -r LINE; do
    if [[ "$LINE" =~ $KEYWORD ]]; then
COUNT=$((COUNT + 1))     fi done <
"$LOGFILE"


# -------- Output Report --------
echo " Log Analysis Report" echo
"File      : $LOGFILE" echo
"Keyword   : $KEYWORD" echo "Count
: $COUNT"


# -------- Show last 5 matches --------
echo "Last 5 Matching Entries:" grep -i
"$KEYWORD" "$LOGFILE" | tail -n 5
```

In open source, errors are just learning moments—tools improve when people find and fix problems.

- **Script 5: The Open Source Manifesto Generator**

Let's make your own FOSS pledge.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
  echo " Generate Your Open Source
Manifesto"


# -------- User Input -------- read -p "1. Name one open-source tool
you use daily: " TOOL read -p "2. What will you build and share with
the community? " BUILD
# -------- Variables --------
DATE_STAMP=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"


# -------- Write Manifesto -------- echo > "$OUTPUT"
echo "OPEN SOURCE MANIFESTO - $DATE_STAMP" >> "$OUTPUT"
echo >> "$OUTPUT"
 echo "I rely on $TOOL as a foundation for my work." >> "$OUTPUT" echo "To
me, software freedom means $FREEDOM." >> "$OUTPUT" echo "I believe
knowledge should be shared, not locked away." >> "$OUTPUT" echo
"Therefore, I commit to building $BUILD and contributing it openly."
>> "$OUTPUT" echo "This is my step towards a collaborative and transparent
future." >>
"$OUTPUT"


# -------- Output --------
echo "Success. Your manifesto is saved at: $OUTPUT"
 cat
"$OUTPUT"
```

Shifting from user to contributor means you're leveling up as a professional.

7. Conclusion: The Foundation of Professionalism

Digging into Git's history shows you open source isn't just a technical option it's the gold standard. The BitKeeper disaster, the power of the GPL and its Four Freedoms, and Git's strong ties to Linux aren't just facts. They shape how we work and think.

- Open Source is how we share knowledge.
- Linux is our bedrock.
- Shell scripts are how we automate everything.

As a systems architect, you need to know both the philosophy and nutsand-bolts of FOSS. Not optional. It's how you keep your work airtight, transparent, and ready for innovation without living at the mercy of vendor whims. Real professionalism means embracing open systems and joining the force that keeps the world running.